

RESEARCHPILOT

Investigation Report

In noisy synthetic regression using random Fourier features, increasing the feature count produces a test-error peak near the interpolation threshold, and ridge regularization reduces that peak.

October 03, 2026

1 Conjecture

In the noisy synthetic regression setting under consideration, the test-error curve of random Fourier feature regression, as a function of feature count with training sample size held fixed, has a peak near the interpolation threshold; using ridge regularization reduces this peak relative to the corresponding unregularized fit. The statement does not specify the data distribution, Fourier-feature distribution, regularization strength, or whether test error denotes a single realization or an average.

Working assumptions: User-given conditions: synthetic regression has observation noise; the fitted representation uses random Fourier features; feature count varies; the comparison concerns test error and ridge regularization., Source-specific conditions, not imposed on the Fourier experiment: the page-19 example uses ReLU features, $\|\beta_1\|_2^2 = 1$, Gaussian noise $\varepsilon_i \sim \mathcal{N}(0, \tau^2)$ with $\tau^2 = 0.2$, and $n/d = 10$. Its optimal ridge parameter is selected separately at each feature count to minimize asymptotic test error., Chosen working implementation: use the cosine features and ridge objective defined above, with the minimum-norm solution as the unregularized baseline. Hold training sample size, target, input distribution, noise level, and kernel bandwidth fixed while increasing feature count., One concrete synthetic test setting can use independent inputs $x_i \sim \mathcal{N}(0, I_d)$, target $f_{-}^*(x) = \beta^{\top} x$ with $\|\beta\|_2^2 = 1$, and independent noise $\varepsilon_i \sim \mathcal{N}(0, \tau^2)$ with $\tau^2 > 0$. These are model-chosen conditions, not specified by the user., For a Gaussian-kernel implementation, choose bandwidth $\ell > 0$ and sample $\omega_j \sim \mathcal{N}(0, \ell^{-2} I_d)$, corresponding to $k(x, x') = \exp\left(-\frac{\|x-x'\|_2^2}{2\ell^2}\right)$. Use prefixes of one sampled feature sequence within each repetition., For reproducible comparison, choose a finite feature-count grid spanning both sides of the observed interpolation threshold and a positive ridge-parameter grid. Interpret the observable curve as repeated-test-error averages. Any data-driven regularization selection uses independent validation data rather than the evaluation test set.

Unverified supporting observations to check: Not specified.

Unverified contradicting observations to check: Not specified.

2 Assessment

ResearchPilot assessment: The conjecture is well supported in the tested Fourier-feature setting, both for the averaged curve and for each observed repetition. The evidence establishes an interpolation-aligned peak and its suppression by three fixed ridge strengths.

Transfer to other synthetic settings or regularization choices remains unresolved. Related literature results were found, but they do not directly support or contradict the conjecture. Experiments support the conjecture in the tested cases.

I interpret the conjecture as describing the noisy synthetic setting under consideration, not as asserting the same behavior for every data distribution or every positive ridge strength. The executed experiment supplies a concrete interpretation: minimum-norm least squares is the unregularized baseline, and test error is MSE against independent noisy labels, averaged over repetitions.

Experiment 1: Fourier-feature interpolation peak and ridge comparison provides the strongest evidence. It held 60 training observations fixed and used six-dimensional standard Gaussian inputs, a first-coordinate linear target, noise variance 0.2, and Gaussian-spectrum cosine features with bandwidth 2.0. Feature counts ranged from 15 to 180. Nested feature prefixes and shared test observations supported paired comparisons. All five repetitions completed without exclusions or reported numerical failures, each using 2,000 test observations.

Every repetition first reached numerical interpolation at 60 features, using a relative residual tolerance of 10^{-8} . Every sampled unregularized curve also attained its maximum at 60 features. The averaged unregularized MSE there was 171.6946, versus 1.9614 at 45 features and 3.6320 at 75 features. Thus the peak location was measured independently through training residuals, rather than assumed from equality between sample size and feature count.

Within the common threshold window of 54, 57, 60, 63, and 66 features, the maxima of the averaged ridge curves were 0.9088, 0.5279, and 0.4559 for $\lambda \in \{0.0001, 0.001, 0.01\}$, respectively. Each ridge strength also reduced each repetition's window maximum. These are distinct comparisons—maxima of averaged curves and paired per-repetition maxima—and both support peak suppression. The fixed-strength results resolve whether feature-count-dependent optimal tuning was necessary in this implementation: it was not.

The inspected page-19 passage of [The generalization error of random features regression: Precise asymptotics and double descent curve](#) adds qualified literature support. Contrary to the earlier review's limitation statement, this passage explicitly describes an interpolation-aligned peak and a comparison with optimal ridge that makes test error strictly decreasing with feature count. However, its example uses ReLU features and separately optimizes the asymptotic error at each feature count. It is supporting context, not a Fourier-feature theorem or a replication of the executed experiment. The inspected page-8 discussion also reports that nonvanishing regularization can hurt at high signal-to-noise ratio. That limits broad claims about ridge improving generalization everywhere; it does not contradict the observed near-threshold reductions.

The remaining limitations concern magnitude and transfer, rather than whether these effects occurred. Unregularized threshold MSE ranged from 11.7865 to 394.0918 across the five repetitions, making the average peak height sampling-sensitive. No confidence interval was supplied. Other targets, distributions, bandwidths, noise levels, sample sizes, ridge strengths, and numerical cutoffs were not tested. Unsampled feature counts and alternative windows could change the measured peak height. Error against the noiseless target was not measured. Finally, the averaged baseline did not initially decrease, so these results support the conjectured interpolation peak, not every aspect of a textbook double-descent curve. None of these scope limits is contradictory evidence against the demonstrated phenomenon.

Direct experimental support: Experiment 1: Fourier-feature interpolation peak and ridge comparison supports both components in its tested setting. It used 60 training observations, six-dimensional standard Gaussian inputs, a first-coordinate linear target, noise variance 0.2,

and Gaussian-spectrum cosine features with bandwidth 2.0. Nested feature prefixes and shared test observations enabled paired comparisons with minimum-norm least squares. All five repetitions completed without reported numerical failures or exclusions.

Every repetition first reached numerical interpolation at 60 features and attained its sampled unregularized test-MSE maximum there. The averaged MSE was 171.6946, versus 1.9614 and 3.6320 at 45 and 75 features. Within the planned window of 54–66 features, fixed ridge strengths $\lambda \in \{0.0001, 0.001, 0.01\}$ reduced the maxima of the averaged curves to 0.9088, 0.5279, and 0.4559. Each strength also reduced each repetition’s window maximum. These are distinct statistics, and both support peak suppression. The averaged baseline did not initially decrease, so the measurements support the stated interpolation peak, not every aspect of a textbook double-descent curve.

Qualified literature support: The inspected page-19 passage of [The generalization error of random features regression: Precise asymptotics and double descent curve](#) explicitly reports an interpolation-aligned peak and decreasing test error under feature-count-dependent optimal ridge. This comparison was missed by the earlier review’s limitation statement. However, the example uses ReLU features and asymptotic-error optimization, not Fourier features with fixed ridge strengths. Its page-8 discussion says nonvanishing regularization can hurt at high signal-to-noise ratio. That qualifies broad claims about ridge improving generalization; it does not contradict the measured near-threshold reductions.

Related findings: The supplied abstract passages

3 Literature evidence

3.1 Supporting evidence

- None verified.

3.2 Contradictory evidence

- None verified.

3.3 Important qualifications

- None verified.

3.4 Related evidence

- As the model complexity increases, the test error follows the usual U-shaped curve at the beginning, first decreasing and then peaking around the interpolation threshold (when the model achieves vanishing training error). [The generalization error of random features regression: Precise asymptotics and double descent curve], section-0, page 1; discussion. Assumptions: The passage describes the double-descent scenario in terms of increasing model complexity., Interpolation is identified with vanishing training error.. This abstract description matches the proposed qualitative peak but supplies no Fourier-feature assumptions, fixed-sample experiment, or proof.
- The paper [KLS18] observes that the optimal amount of ridge regularization is sometimes vanishing, and provides an explanation in terms of noisy features. [The generalization error of random features regression: Precise asymptotics and double descent

curve], section-0, page 9; discussion. Assumptions: The observation concerns the setting of the cited work and is explained in terms of noisy features.. This second-hand literature discussion cautions against assuming that positive ridge strength is always optimal. It does not decide whether ridge reduces a near-interpolation peak in the conjectured F

- Based on the calculation, we further theoretically demonstrate that the risk curves of DRFMs can exhibit triple descent. [Multiple Descent in the Multiple Random Feature Model], section-0, page 1; related_result. Assumptions: The double random feature model concatenates two types of random features., The estimator uses ridge regression., Training sample size, input dimension, and random-feature dimension tend to infinity proportionally.. The abstract reports a related risk-curve phenomenon, not a supplied theorem or proof. Multiple feature components differ from the unspecified Fourier-feature construction, and no peak-reduction compa
- We take 100 instances of random training data sets, and for each we test on 500 samples. We report the average test error over all 50,000 test cases. [High-Dimensional Asymptotics of Prediction: Ridge Regression and Classification], section-0, page 11; discussion. Assumptions: The caption concerns ridge regression in the BinaryTree and Exponential models., Signals are drawn from $w \sim N(0, p - 1I_{p \times p})$., Test error is averaged across 100 training datasets and 500 test samples per dataset.. This supplies relevant context for distinguishing averaged test error from a single realization. It reports an evaluation protocol, not measured evidence of either conjectured effect.
- We derive a result based on the behavior of the smallest singular value of the regression matrix that explains the peak location and the double descent shape of the testing error as a function of model order. [Analysis of Interpolating Regression Models and the Double Descent Phenomenon], section-0, page 1; discussion. Assumptions: The abstract focuses on interpolating models derived from the minimum-norm solution to classical least squares., The curve varies model order.. The abstract identifies a potentially relevant mechanism but supplies neither the mathematical result nor its proof. It does not establish applicability to random Fourier features or ridge-induced pea

4 Computational investigation

Experiments planned: 1. Executed successfully: 1. Stochastic trials: 0.

4.1 Experiment 1: Fourier-feature interpolation peak and ridge comparison

Test type: falsifying. **Baselines:** Minimum-norm least squares **Metrics:** Test MSE, in squared-label units: $E_{\{s, \lambda\}}(m) = \frac{1}{n_{\{\text{test}\}}} \sum_{i=1}^{n_{\{\text{test}\}}} \left(\phi_{\{s, m\}}(x_{\{\text{test}\}, s, i}) - y_{\{\text{test}\}, s, i} \right)^2$. Average over completed repetitions: $\bar{E}_{\lambda}(m) = \frac{1}{S} \sum_{s=1}^S E_{s, \lambda}(m)$. Report the completed-repetition count S ., Numerical interpolation threshold, in features: $m_{\{\text{int}\}, s} = \min \left\{ m : \frac{\|\Phi_{\{s, m\}} \Phi_{\{s, m\}}^{\dagger} y_s - y_s\|_2}{\|\Phi_{\{s, m_{\{\text{int}\}, s}\}} \Phi_{\{s, m_{\{\text{int}\}, s}\}}^{\dagger} y_s - y_s\|_2} \leq \epsilon_{\{\text{int}\}} \right\}$, restricted to threshold_scan and using svd_relative_cutoff. Report each threshold or 'not reached'; if all completed repetitions reach it, report $\tilde{m}_{\text{int}} = \text{median}_{s=1, \dots, S} m_{\text{int}, s}$. Define the near-threshold grid window by $\mathcal{W} = \left\{ m \in \mathcal{M} : \left| \frac{m}{\tilde{m}_{\text{int}}} - 1 \right| \leq \delta \right\}$, where $\mathcal{M}$ is feature_counts and δ is near_threshold_fraction. Define adjacent flank counts by

$m_- = \max\{m \in \mathcal{M} : m < \min \mathcal{W}\}$ and $m_+ = \min\{m \in \mathcal{M} : m > \max \mathcal{W}\}$., Near-threshold peak MSE and unregularized peak prominence, in squared-label units: $P_\lambda = \max_{m \in \mathcal{W}} \bar{E}_\lambda(m)$ and $C_0 = P_0 - \max\{\bar{E}_0(m_-), \bar{E}_0(m_+)\}$. Compute these from the averaged curves., Ridge peak reduction, in squared-label units, separately for every positive setting in `ridge_parameters`: $D_\lambda = P_0 - P_\lambda$. Report window-based metrics as unavailable if a threshold, the window, or either flank is unavailable.

Algorithm pseudocode:

1. Start the wall-clock timer and enforce `linear_algebra_threads` and `arithmetic`.
2. Loop over `seeds`, checking the work-cap deadline before each expensive operation; discard an unfinished repetition if the deadline is reached.
3. Initialize an independent random stream from the current seed.
4. Generate independent training and test inputs from `input_distribution` with dimensions specified by `n_train`, `n_test`, and `d`.
5. Generate independent observation noise with $\varepsilon_i \sim \mathcal{N}(0, \tau^2)$, where τ^2 is `noise_variance`, and form both datasets using $y_i = x_{i,1} + \varepsilon_i$.
6. Sample the maximum permitted feature sequence independently of the data: $\omega_j \sim \mathcal{N}(0, \ell^{-2} I_d)$ and $b_j \sim \text{Uniform}[0, 2\pi)$, where ℓ is `kernel_bandwidth`.
7. Cache training and test cosine values using $C_{\{ij\}} = \cos(\omega_j^{\text{top}} x_i + b_j)$.
8. Scan feature counts in `threshold_scan` in ascending order, constructing each normalized prefix with $\Phi_m = \sqrt{\frac{2}{m}} C_{:,1:m}$.
9. For each scanned prefix, compute a thin singular-value decomposition and construct its pseudoinverse by retaining singular values satisfying $\sigma_k > \rho \sigma_1$, where ρ is `svd_relative_cutoff`.
10. Record the first prefix satisfying the interpolation-residual condition in `metrics` and stop the threshold scan; record 'not reached' if the scan ends without satisfying it.
11. Loop over `feature_counts` and construct the corresponding normalized training and test prefixes.
12. Compute the training-prefix thin singular-value decomposition $\Phi_m = U \Sigma V^{\text{top}}$.
13. Fit the minimum-norm least-squares baseline using $\widehat{a}_{\{m,0\}} = V \Sigma^{\{\dagger\}} U^{\text{top}} y$, with the same singular-value cutoff used in the threshold scan.
14. Loop over `ridge_parameters` and fit the specified ridge objective using $\widehat{a}_{\{m,\lambda\}} = V \text{operatorname{diag}} \left(\frac{1}{\sigma_k^2 + n_{\{\text{train}\}} \lambda} \right) U^{\text{top}} y$, without truncating positive singular values for ridge.
15. Evaluate every fitted predictor on the same independent test observations and store its test MSE as defined in `metrics`.
16. Retain the repetition only after all its threshold and curve measurements are complete.
17. Aggregate the completed repetitions according to `metrics`, construct the threshold window and flanks when defined, and report all baseline and ridge measurements.

Working assumptions:

- User-given conditions: synthetic regression has observation noise; the fitted representation uses random Fourier features; feature count varies; the comparison concerns test error and ridge regularization.

- Source-specific conditions, not imposed on the Fourier experiment: the page-19 example uses ReLU features, $\|\beta_1\|_2^2 = 1$, Gaussian noise $\varepsilon_i \sim \mathcal{N}(0, \tau^2)$ with $\tau^2 = 0.2$, and $n/d = 10$. Its optimal ridge parameter is selected separately at each feature count to minimize asymptotic test error.
- Chosen working implementation: use the cosine features and ridge objective defined above, with the minimum-norm solution as the unregularized baseline. Hold training sample size, target, input distribution, noise level, and kernel bandwidth fixed while increasing feature count.
- One concrete synthetic test setting can use independent inputs $x_i \sim \mathcal{N}(0, I_d)$, target $f_*(x) = \beta^{\top} x$ with $\|\beta\|_2^2 = 1$, and independent noise $\varepsilon_i \sim \mathcal{N}(0, \tau^2)$ with $\tau^2 > 0$. These are model-chosen conditions, not specified by the user.
- For a Gaussian-kernel implementation, choose bandwidth $\ell > 0$ and sample $\omega_j \sim \mathcal{N}(0, \ell^{-2} I_d)$, corresponding to $k(x, x') = \exp\left(-\frac{\|x-x'\|_2^2}{2\ell^2}\right)$. Use prefixes of one sampled feature sequence within each repetition.
- For reproducible comparison, choose a finite feature-count grid spanning both sides of the observed interpolation threshold and a positive ridge-parameter grid. Interpret the observable curve as repeated-test-error averages. Any data-driven regularization selection uses independent validation data rather than the evaluation test set. **Exact planned settings:**
 - held constant: `{"n_train":60, "n_test":2000, "d":6, "input_distribution":"standard multivariateGaussian", "target":"first input coordinate", "noise_variance":0.2, "kernel_bandwidth":2.0, "phase_distribution":"uniform on [0, 2 π)", "feature_sequence":"nested prefixes within each repetition", "test_labels":"independentnoisy labels", "linear_algebra_threads":1, "arithmetic":"float64", "svd_relative_cutoff":1e-12, "interpolation_residual_tolerance":1e-08, "near_threshold_fraction":0.1}`
 - varied parameters: `{"feature_counts":[15, 30, 45, 54, 57, 60, 63, 66, 75, 90, 120, 180], "ridge_parameters":[0.0001, 0.001, 0.01]}`
 - baseline regularization: 0
 - threshold scan: `{"first_feature_count":1, "last_feature_count":180, "increment":1}`
 - stopping rule: "Finish all listed repetitions and settings, or stop at the wall-clock deadline; aggregate only completed repetitions and report their count. If none complete, report no measurements."
 - work cap: `{"wall_clock_seconds":280, "maximum_features_per_repetition":180, "maximum_threshold_svd_calls_per_repetition":180, "maximum_curve_svd_calls_per_repetition":12, "maximum_coefficient_fits_per_repetition":48}`
 - random seeds: [17, 43, 89, 131, 197].

Execution: completed; run experiment_d9248b8453cc; runtime 38.60 seconds.

Recorded configuration: `{"held_constant":{"n_train":60, "n_test":2000, "d":6, "input_distribution":"standard multivariate Gaussian", "target":"first input coordinate", "noise_variance":0.2, "kernel_bandwidth":2.0, "phase_distribution":"uniform on [0, 2 π)", "feature_sequence":"nested prefixes within each repetition", "test_labels":"independentnoisy labels", "linear_algebra_threads":`

```
1, "arithmetic": "float64", "svd_relative_cutoff": 1e-12, "interpolati
on_residual_tolerance": 1e-08, "near_threshold_fraction": 0.1},
"varied_parameters": {"feature_counts": [15, 30, 45, 54, 57, 60, 63, 66,
75, 90, 120, 180], "ridge_parameters": [0.0001, 0.001, 0.01]}, "baseline_
regularization": 0, "threshold_scan": {"first_feature_count": 1, "last_
feature_count": 180, "increment": 1}, "stopping_rule": "Finish all
listed repetitions and settings, or stop at the wall-clock deadline;
aggregate only completed repetitions and report their count. If
none complete, report no measurements.", "work_cap": {"wall_clock_
seconds": 280, "maximum_features_per_repetition": 180, "maximum_
threshold_svd_calls_per_repetition": 180, "maximum_curve_svd_calls_
per_repetition": 12, "maximum_coefficient_fits_per_repetition": 48}}
```

Recorded metrics: {"experiment": "Fourier-feature interpolation peak and ridge comparison", "method": "Gaussian-spectrum cosine features; direct Golub-Reinsch thin SVD; cutoff minimum-norm least squares and untruncated SVD ridge", "source_scope": "The supplied paper example uses ReLU and feature-dependent optimal ridge; this is the explicitly designed Fourier experiment with fixed ridge strengths, not a replication of that example.", "settings": {"n_train": 60, "n_test": 2000, "d": 6, "input_distribution": "standard multivariate Gaussian", "target": "first input coordinate", "noise_variance": 0.2, "kernel_bandwidth": 2.0, "phase_distribution": "uniform on [0, 2pi)", "feature_sequence": "nested prefixes within each repetition", "test_labels": "independent noisy labels", "linear_algebra_threads": 1, "arithmetic": "float64", "svd_relative_cutoff": 1e-12, "interpolati on_residual_tolerance": 1e-08, "near_threshold_fraction": 0.1, "feature_counts": [15, 30, 45, 54, 57, 60, 63, 66, 75, 90, 120, 180], "ridge_ parameters": [0.0001, 0.001, 0.01], "baseline_regularization": 0, "ridge_ objective": " $||\Phi a - y||^2/n_{\text{train}} + \lambda ||a||^2$ ", "threshold_ scan": {"first_feature_count": 1, "last_feature_count": 180, "increment": 1}, "work_cap": {"wall_clock_seconds": 280, "maximum_features_per_ repetition": 180, "maximum_threshold_svd_calls_per_repetition": 180, "maximum_curve_svd_calls_per_repetition": 12, "maximum_coefficient_ fits_per_repetition": 48}, "rng": "stdlib random.Random, Gaussian draws via gauss; one independent stream per seed"}, "requested_ seeds": [17, 43, 89, 131, 197], "completed_seeds": [17, 43, 89, 131, 197], "completed_repetition_count": 5, "discarded_or_unstarted_seeds": [], "deadline_reached": false, "numerical_failure": null, "elapsed_ seconds": 31.354512481999997, "repetitions": [{"seed": 17, "threshold": 60, "threshold_censored": false, "threshold_scan_m_residual_rank": [[1, 0.9868344288007893, 1], [2, 0.7896287901731258, 2], [3, 0.7881938024 324125, 3], [4, 0.7681001177918559, 4], [5, 0.7589114803371395, 5], [6, 0.7356383930731559, 6], [7, 0.7316381001533837, 7], [8, 0.7276495918 51299, 8], [9, 0.7162369082659981, 9], [10, 0.6885541508586287, 10], [11, 0.6875892263167603, 11], [12, 0.5024583026106855, 12], [13, 0.4942244945907997, 13], [14, 0.4901803639704515, 14], [15, 0.4755568220 3200494, 15], [16, 0.47185712313656397, 16], [17, 0.45764054468151744, 17], [18, 0.44064764365233383, 18], [19, 0.44054199597617755, 19], [20, 0.43861283378832194, 20], [21, 0.4351722331202802, 21], [22, 0.4278999756584882, 22], [23, 0.4268135832792341, 23], [24, 0.4262863996 928958, 24], [25, 0.41835974463582903, 25], [26, 0.41779452789864513,

```
26], [27, 0.4172137196716295, 27], [28, 0.4171837065013468, 28], [29,
0.40847612347403206, 29], [30, 0.4048419298059397, 30], [31, 0.3959156685
3682064, 31], [32, 0.38748593400872783, 32], [33, 0.38740904310911056,
33], [34, 0.3749985136415035, 34], [35, 0.3709804151146434, 35],
[36, 0.370092869685531, 36], [37, 0.36282349923267787, 37], [38,
0.35338737171897316, 38], [39, 0.3481947349084725, 39], [40, 0.2990470075
20402, 40], [41, 0.2990465758196102, 41], [42, 0.24996087622020935,
42], [43, 0.24271253951267227, 43], [44, 0.24192321523166088, 44],
[45, 0.22543869588686166, 45], [46, 0.21040353795406014, 46], [47,
0.2102373297926166, 47], [48, 0.20321125420769862, 48], [49, 0.2029205085
3894236, 49], [50, 0.20262579419588128, 50], [51, 0.19929312456451626,
51], [52, 0.19843523232026702, 52], [53, 0.19626448250069387, 53], [54,
0.1457303392863154, 54], [55, 0.1451616078658758, 55], [56, 0.1362155334
6467047, 56], [57, 0.13621317713752645, 57], [58, 0.11913303703158896,
58], [59, 0.02385944858244533, 59], [60, 3.338483224668617e-14, 60]],
"test_mse": {"0": [0.6093916844578774, 0.7230191339488392, 3.2003412266
548192, 10.748629820013699, 8.843409940228062, 111.27135274300886,
64.76815377602193, 6.292868894092625, 3.1493425069992873, 2.2264979225
225523, 1.5097924697287468, 1.3623124066170966], "0.0001": [0.605482733
9473456, 0.7146768383180209, 0.8711015064025922, 1.1686431565
84764, 0.947358227283801, 0.9170417647701462, 0.8679091445408484,
0.883979664919387, 0.8554171179438504, 0.8908149074858861,
0.7642588954136023, 0.8624565293882164], "0.001": [0.581067848
1210155, 0.6605579136436875, 0.5420367620730941, 0.5048696279
661659, 0.45409987118908524, 0.445219755727352, 0.4386170669
6761543, 0.4427534753255407, 0.4353066749258478, 0.4035340174709676,
0.38329456697509623, 0.4405598176506798], "0.01": [0.536464091
2629668, 0.571762285532313, 0.5151093133700106, 0.4605588184436772,
0.42712058922845336, 0.41777202117438755, 0.41951472284901586,
0.4145873819003632, 0.39494269472070664, 0.36863465804259554,
0.36292375569823654, 0.36691800746331454]}, "baseline_relative_train_
residual": [0.47555682203200494, 0.4048419298059397, 0.2254386958
8686166, 0.1457303392863154, 0.13621317713752645, 3.338483224668617e-
14, 1.2742108211918328e-14, 3.9201150130738995e-15, 2.4373190613
24648e-15, 1.7080189171356976e-15, 1.9407628255522167e-15,
1.7010312616575468e-15], "curve_singular_max_min": [[4.57894587
6189532, 0.32350376106155404], [4.295208751784393, 0.1416732679
9078706], [4.3944494790662, 0.03364726411103908], [4.1603150031487255,
0.01033217168512515], [4.129034691416314, 0.004542385180678049],
[4.1498273088850395, 0.001757166400769543], [4.111037888116305,
0.002116521620017113], [4.080787836959318, 0.010904507351559106],
[4.129483098956111, 0.016511284461508496], [4.139664772429505,
0.027293852159920334], [4.365457553042853, 0.04329914496197025],
[4.456347392537475, 0.0465697946475878]], "work_counts": {"threshold_
svd_calls": 60, "curve_svd_calls": 12, "curve_coefficient_fits": 48}},
{"seed": 43, "threshold": 60, "threshold_censored": false, "threshold_
scan_m_residual_rank": [[1, 0.9847480642145653, 1], [2, 0.9794766532
42457, 2], [3, 0.9734968468467136, 3], [4, 0.8929073446349569, 4], [5,
0.890237763063796, 5], [6, 0.8683621018536405, 6], [7, 0.8453173170
655168, 7], [8, 0.8264862523883177, 8], [9, 0.823601904466871, 9], [10,
0.82222786748773, 10], [11, 0.8221921326980476, 11], [12, 0.5964988561
```

```
993962,12],[13,0.571133493260743,13],[14,0.5462168642564985,
14],[15,0.5268629022406404,15],[16,0.513831621743345,16],[17,
0.5069428101624552,17],[18,0.4901796407103066,18],[19,0.4895992912
5488233,19],[20,0.47808412190278904,20],[21,0.46954343526795156,
21],[22,0.4693788344928536,22],[23,0.4277489533249428,23],[24,
0.39346414703561994,24],[25,0.3859696580293942,25],[26,0.3778369516
279314,26],[27,0.362897078130738,27],[28,0.3622274616900346,
28],[29,0.3622114679598841,29],[30,0.3447860657319981,30],[31,
0.3442604131136855,31],[32,0.33120027574917654,32],[33,0.3104145247
3735365,33],[34,0.30557131721739544,34],[35,0.3011708101179304,
35],[36,0.2986310934582035,36],[37,0.2979485686100677,37],[38,
0.2967257421816271,38],[39,0.29168364909835354,39],[40,0.2709985520
9867435,40],[41,0.26293735171573546,41],[42,0.24782021238048293,
42],[43,0.24726750254884608,43],[44,0.24579855856903693,44],[45,
0.2416566663243036,45],[46,0.20031200858434428,46],[47,0.2002553713
1272908,47],[48,0.20016943471092288,48],[49,0.1987862166401485,
49],[50,0.16169678417699973,50],[51,0.16095750616594698,51],[
52,0.15824645217628042,52],[53,0.15824275715337544,53],[54,
0.15538611307591405,54],[55,0.1477563775729832,55],[56,0.1477356191
6877312,56],[57,0.13944446911546568,57],[58,0.09528039174095855,
58],[59,0.09190787042042886,59],[60,2.696564114907243e-14,60]],
"test_mse":{"0":[0.7839348272642679,0.625509816812327,0.8315992826
174765,3.19831837874375,13.83923511685859,279.6641804103129,
84.97058038202381,20.100887291344506,4.692379803781918,3.7345308087
24541,3.1088737804789526,1.6449983557638566],"0.0001":[0.765694073
4382196,0.6004180397427454,0.6188710035832063,0.6370985523
49258,0.6461998031296512,0.6540366962419712,0.6285523964
205181,0.603868520367972,0.7221933303506879,0.7002645801790345,
0.6865161293414697,0.7044200734920779],"0.001":[0.660575798
4934983,0.5101676025156539,0.4822571943750251,0.4759168470
3824816,0.47336438028980465,0.45928599624509153,0.4511528042
435016,0.44386268751320407,0.4623997667661092,0.4464981294598136,
0.4565833231537416,0.45523166425176603],"0.01":[0.611965411
9243084,0.4184414580779405,0.4161566944804978,0.4116867835
2693355,0.41841447837601503,0.41460918746116326,0.4132281637
281667,0.41591132245498624,0.4077360321203164,0.3981748282
977525,0.3962856967534336,0.39568831484614553]},"baseline_
relative_train_residual":[0.5268629022406404,0.3447860657319981,
0.2416566663243036,0.15538611307591405,0.13944446911546568,
2.696564114907243e-14,1.6617699661147777e-14,7.741325744932809e-
15,3.849210166440532e-15,3.947277297622157e-15,2.6918923828
083444e-15,2.1949434515433857e-15],"curve_singular_max_min":
[[5.021440448900432,0.4377381674621686],[4.236211841912338,
0.13104988295597175],[4.310258152489808,0.0357723730179653],
[4.208053215627089,0.00931360754999381],[4.208467163949968,
0.0037757054136348367],[4.273004322681244,0.001044635910642036]
,[4.3476398605033735,0.002364114392826365],[4.4167701477285,
0.004365766700439431],[4.353153680222874,0.014281279667636747]
,[4.416817285841908,0.02092562320310844],[4.2620940413912605,
0.033783570164871705],[4.435003016745772,0.04895611289565247]
],"work_counts":{"threshold_svd_calls":60,"curve_svd_calls":
```

```
12, "curve_coefficient_fits":48}}, {"seed":89, "threshold":60,
"threshold_censored":false, "threshold_scan_m_residual_rank":[[1,
0.9770915057353791, 1], [2, 0.9725284556163365, 2], [3, 0.9339152013
645415, 3], [4, 0.9315342912088845, 4], [5, 0.926464260620494, 5], [6,
0.9217620142160216, 6], [7, 0.9199104211707698, 7], [8, 0.9089542727
670694, 8], [9, 0.9076310926997643, 9], [10, 0.9045714532411143, 10]
, [11, 0.9044501814134305, 11], [12, 0.9006412946001907, 12], [13,
0.9005383547608042, 13], [14, 0.7955350177808019, 14], [15, 0.7802848033
59382, 15], [16, 0.7801467800181202, 16], [17, 0.7796335302949646,
17], [18, 0.778776381885438, 18], [19, 0.6826555088067758, 19], [20,
0.5489418973702052, 20], [21, 0.5234794466716383, 21], [22, 0.5226858553
552236, 22], [23, 0.5060739926374767, 23], [24, 0.40808647940257897,
24], [25, 0.4058430190692728, 25], [26, 0.3788499647282814, 26], [27,
0.3783938314312279, 27], [28, 0.3707157252026852, 28], [29, 0.3471493613
352364, 29], [30, 0.3285246583808253, 30], [31, 0.3253736324631938,
31], [32, 0.32534694942069275, 32], [33, 0.30583166223342156, 33], [34,
0.2842333075739038, 34], [35, 0.28283336021908473, 35], [36, 0.2794093846
49086, 36], [37, 0.25204124930778504, 37], [38, 0.2449525791405847,
38], [39, 0.24315041844053012, 39], [40, 0.24159356323725337, 40], [41,
0.20442106622458894, 41], [42, 0.2040971337848658, 42], [43, 0.2018574760
2658433, 43], [44, 0.19934468053222704, 44], [45, 0.19934397099328174,
45], [46, 0.19874022755988469, 46], [47, 0.16374397249761155, 47],
[48, 0.16255049579615677, 48], [49, 0.1586042307146402, 49], [50,
0.13140014884199125, 50], [51, 0.1309930271135211, 51], [52, 0.1270051337
5218114, 52], [53, 0.12684412874706155, 53], [54, 0.09155515975321614,
54], [55, 0.08078027305287118, 55], [56, 0.08058154065197676, 56],
[57, 0.04751924696207726, 57], [58, 0.037266193236296624, 58], [59,
0.03710856124618964, 59], [60, 1.299838904795392e-14, 60]], "test_
mse":{"0":[0.9106690035350387, 1.1993765150868427, 1.8993576778
953767, 8.805513777396342, 15.98012283383831, 61.65931017245382,
9.111914676523618, 6.499176889059844, 5.498871927760535, 3.5747827125
619063, 2.5454374841217207, 2.136286757540468], "0.0001":[0.909339994
8091562, 1.0949913599181955, 1.1543893680845994, 1.1909896193
068208, 1.2633557383631406, 1.2005067084972931, 1.0945351258
465417, 1.0690148588127302, 0.9345721369198969, 0.9347044131610436,
0.9025601986214102, 0.9315540169344045], "0.001":[0.901526917
1630846, 0.8116916886209584, 0.7260906315866889, 0.7038630041
703043, 0.6792876455413924, 0.6453308240116958, 0.6031843640
32144, 0.6058115551523331, 0.574701863795957, 0.5423102644305188,
0.5104668735749324, 0.4909992861570073], "0.01":[0.913845566
8159602, 0.7106172445829515, 0.6189399759665574, 0.5666793448
507286, 0.5343873418757948, 0.5236538435697803, 0.5054891015
543971, 0.5065008946483052, 0.5018565227263644, 0.4770992860
038589, 0.4675832839307553, 0.43795135087454756]}, "baseline_
relative_train_residual":[0.780284803359382, 0.3285246583808253,
0.19934397099328174, 0.09155515975321614, 0.04751924696207726,
1.299838904795392e-14, 7.773625674926187e-15, 3.1958599263849753e-
15, 4.728200013871377e-15, 2.6691805992306204e-15, 1.8271890043
19348e-15, 2.2309782116123774e-15], "curve_singular_max_min":
[[4.950305908978277, 0.3387364242620858], [4.382622587264238,
0.13620880330688265], [4.2682125247913145, 0.022399864108407226]
```

```
, [4.193016427650723, 0.007563111227389375], [4.120846438045752,
0.006831255329695033], [4.078264557327733, 0.000815675723219072]
, [4.094196487747592, 0.005032808443880384], [4.142079975508511,
0.01062455367996917], [4.291922841298547, 0.016422852400242924]
, [4.375267069877219, 0.023785914181017206], [4.449197172762833,
0.02914318893964505], [4.39821232293451, 0.05241384635501004]],
"work_counts":{"threshold_svd_calls":60,"curve_svd_calls":12,
"curve_coefficient_fits":48}}, {"seed":131,"threshold":60,"threshold_
censored":false,"threshold_scan_m_residual_rank":[[1,0.9791859989
081562,1],[2,0.9781484391729558,2],[3,0.9775117902494842,3],[4,
0.9146767908980402,4],[5,0.8095718733858894,5],[6,0.8021948307
218756,6],[7,0.7829600787658362,7],[8,0.7746222892097885,8],[9,
0.7351706441679978,9],[10,0.7212081874113684,10],[11,0.7125627313
530606,11],[12,0.665073340605677,12],[13,0.6560133814239212,
13],[14,0.5709545409320901,14],[15,0.5701549215251668,15],[16,
0.5697415838662798,16],[17,0.5685377168246456,17],[18,0.5588672201
88913,18],[19,0.5574813769982171,19],[20,0.5568898793435857,
20],[21,0.5568851089378613,21],[22,0.5553278284042922,22],[23,
0.4777047632219478,23],[24,0.47266559763450683,24],[25,0.4679343495
9303326,25],[26,0.466813136582978,26],[27,0.45047359174807433,
27],[28,0.45033085142627216,28],[29,0.44995468528141147,29],[30,
0.36946665307288507,30],[31,0.350344615024832,31],[32,0.3419528918
6381417,32],[33,0.34163442897397894,33],[34,0.33807914385744864,
34],[35,0.3102328289493799,35],[36,0.30543701387435196,36],[37,
0.30542614191487,37],[38,0.30485599549932196,38],[39,0.2989250178
6293113,39],[40,0.2723654187714763,40],[41,0.2434413104175451,
41],[42,0.24101588165112645,42],[43,0.1808751403753107,43],
[44,0.17806817989462032,44],[45,0.17513310683358405,45],[46,
0.1543361921984057,46],[47,0.1501721683939114,47],[48,0.1478631994
7775647,48],[49,0.12600315128979087,49],[50,0.10940826940684395,
50],[51,0.10428842770213716,51],[52,0.09870483120974538,52]
,[53,0.09064895341851424,53],[54,0.08594492973753663,54],
[55,0.08556265987600883,55],[56,0.08551037628330939,56],[57,
0.08549216533361412,57],[58,0.06266598842719162,58],[59,
0.02992717976455616,59],[60,4.8176548370421123e-14,60]], "test_
mse":{"0":[0.8717941031295451,0.7680989514037853,2.4801074834
521746,3.2711996756319253,2.864351384074433,394.09176581393183,
6.959247063934116,4.732490323505585,2.004360836338447,1.1693572972
753243,0.8978654041852854,0.776102503729662],"0.0001":[0.858657959
9131962,0.7101197546361958,0.7178205501591737,0.6884042509
218063,0.7245374285298689,0.7661730295633397,0.8087134215
216752,0.7919028995278415,0.6988543372249052,0.6247786673840369,
0.5669582931633653,0.5346142104713495],"0.001":[0.771391776
5927241,0.5431082606458801,0.46651870794943745,0.4230685597
476556,0.4294454122496108,0.4534185077372684,0.45709831054454353,
0.44240795725747595,0.4239678408281918,0.42351869396566294,
0.4227912665190684,0.40502610729609806],"0.01":[0.607357331
1537614,0.49390400671304424,0.45619779485457174,0.3978768487
077916,0.41023543240667576,0.41720519124545985,0.4192192285
6196764,0.40587834959621066,0.4032291434374836,0.4021685593
5968715,0.3867417872481266,0.3671123275948314]}},"baseline_
```

```
relative_train_residual": [0.5701549215251668, 0.36946665307288507,
0.17513310683358405, 0.08594492973753663, 0.08549216533361412,
4.8176548370421123e-14, 4.6014506748951065e-15, 2.7524071418013345e-
15, 3.120726973281355e-15, 2.271887317545815e-15, 1.5230622787934508e-
15, 2.0121229650653555e-15], "curve_singular_max_min": [[5.17918964
1657496, 0.458403789984984], [4.907488045345199, 0.1037736157515263]
, [4.624547642571037, 0.03196876927525671], [4.388494489897248,
0.010646228877644085], [4.318685478079848, 0.0032271477091942646]
, [4.315968299765281, 0.00045392864890114525], [4.280234589870226,
0.004563428163712943], [4.3940856253759755, 0.006286390449669415]
, [4.427449955871778, 0.009498992580927788], [4.396269018403207,
0.017516649433204627], [4.368084636551685, 0.0290201404031711],
[4.448376293233227, 0.04502559792916634]], "work_counts": {"threshold_
svd_calls": 60, "curve_svd_calls": 12, "curve_coefficient_fits": 48}},
{"seed": 197, "threshold": 60, "threshold_censored": false, "threshold_
scan_m_residual_rank": [[1, 0.8358768111838235, 1], [2, 0.8255346608
515417, 2], [3, 0.8253022396254229, 3], [4, 0.7972817923395416, 4], [5,
0.7919466996755399, 5], [6, 0.7840696721104279, 6], [7, 0.7815860581
866519, 7], [8, 0.7042923730372472, 8], [9, 0.6982398093902171, 9]
, [10, 0.6943406452298437, 10], [11, 0.6941698063890304, 11], [12,
0.6700936622810372, 12], [13, 0.6699913383520157, 13], [14, 0.6686815672
81163, 14], [15, 0.667360204782925, 15], [16, 0.6567666720027533, 16]
, [17, 0.6547348602900412, 17], [18, 0.6544208228911538, 18], [19,
0.617241711646029, 19], [20, 0.5888198865711696, 20], [21, 0.5863380300
464991, 21], [22, 0.5538016844789878, 22], [23, 0.38088186581358696,
23], [24, 0.37348069191846195, 24], [25, 0.3682219543324494, 25], [26,
0.3553891849812815, 26], [27, 0.3141720737191953, 27], [28, 0.3068083030
9215396, 28], [29, 0.301408196687803, 29], [30, 0.29112083623789275,
30], [31, 0.28590739433526136, 31], [32, 0.28276368150139614, 32], [33,
0.276474171279126, 33], [34, 0.26934823622697923, 34], [35, 0.2691580097
5347437, 35], [36, 0.25255856072550925, 36], [37, 0.24570430203387972,
37], [38, 0.23484182706216797, 38], [39, 0.23456322149345063, 39]
, [40, 0.21089451634745315, 40], [41, 0.21080329090976643, 41],
[42, 0.19838346707472387, 42], [43, 0.19418410089882293, 43], [44,
0.18053783830282444, 44], [45, 0.1782811080125646, 45], [46, 0.1425572105
9799414, 46], [47, 0.13251253894943538, 47], [48, 0.13200918684326246,
48], [49, 0.13200694713199626, 49], [50, 0.12562404554611398, 50]
, [51, 0.11233019265153192, 51], [52, 0.09519384999363875, 52],
[53, 0.062359770387782475, 53], [54, 0.05604294323678307, 54],
[55, 0.055586343764549594, 55], [56, 0.03632413533285752, 56],
[57, 0.03557154026306253, 57], [58, 0.01954915638092458, 58], [59,
0.01946679385644271, 59], [60, 1.0616613480656454e-14, 60]], "test_
mse": {"0": [0.919200114026833, 0.8376727719423795, 1.3958223619
65631, 5.6452228357520235, 6.279690466974648, 11.786481290326796,
7.727491058693748, 6.120052421238772, 2.814811745718329, 2.5653597493
30958, 1.8042944375928929, 1.4601973758332], "0.0001": [0.915917231
3590412, 0.7703588676735387, 0.7674681094057325, 0.8588402910
845739, 0.8478633011273573, 0.9039361267159962, 0.8918885464
809797, 0.9190390175107325, 0.9030030623817482, 1.002937022960439,
0.8993655730223699, 0.8097003041046557], "0.001": [0.890863080
9762254, 0.6330595624962365, 0.5749849066768126, 0.5320232387
```

```
781104,0.5204740504631309,0.5534432431623894,0.5483694093
983419,0.5544173255939362,0.5423375683259061,0.5289006523785966,
0.4825616583321035,0.4647213694801534], "0.01": [0.8047241100000064,
0.49315172708286614,0.5013412698932961,0.4426338107557946,
0.4323532573126044,0.44154014168637223,0.44337277375546297,
0.44159947065127064,0.44266767472087787,0.4277066067864975,
0.4043175479153092,0.41051241090752766]], "baseline_relative_train_
residual": [0.667360204782925,0.29112083623789275,0.1782811080
125646,0.05604294323678307,0.03557154026306253,1.0616613480656454e-
14,5.460245011247908e-15,3.8882240020682876e-15,1.9776294889
68773e-15,2.5378859480331756e-15,1.4639422375748095e-15,
2.0327514143879263e-15], "curve_singular_max_min": [[4.36792821
638948,0.6503316719147009], [4.702560172141698,0.10168886290142726]
, [4.638344419067994,0.02839520654418568], [4.720232091109275,
0.007698887068991425], [4.668887890972726,0.006433407763283685]
, [4.728236818816796,0.0019490283394266819], [4.791759318474946,
0.005628663871557848], [4.827939021903124,0.008954918018462007]
, [4.825783770060771,0.014694430801774783], [4.857988235733104,
0.027517577832749225], [4.772399691924354,0.029821640900159114],
[4.672924263414783,0.0395719165176609]], "work_counts": {"threshold_
svd_calls": 60, "curve_svd_calls": 12, "curve_coefficient_fits": 48}
}], "average_test_mse": {"0": [0.8189979464827124,0.8307354378
388346,1.9614456065170955,6.333776897507548,9.561361948394808,
171.69461808600684,34.70747739143944,8.749095163848267,3.6319533641
197035,2.6541056980830566,1.9732527152215198,1.4759794798968566]
, "0.0001": [0.8110183986933917,0.7781129720577392,0.8259301075
270609,0.9087951740494447,0.8858628996867639,0.8883388651
577493,0.8583197269621126,0.8535609922277325,0.8228079969
642177,0.830699918234088,0.7639318179124435,0.7685490268781408]
, "0.001": [0.7610850842693095,0.6317170055844833,0.5583776405
322116,0.5279482555400968,0.5113342719466047,0.5113396653767595,
0.49968439103722934,0.497850600168498,0.48774274292840236,
0.4689523515411119,0.4511395377109884,0.45130764896714093],
"0.01": [0.6948713022314006,0.5375753443978231,0.5015490097129868,
0.45588712125698516,0.44450221983990873,0.44295607702743267,
0.440164798089802,0.43689548385022714,0.43008641354514976,
0.41475678769807833,0.40357041430917223,0.39563648233727333]],
"mse_units": "squared-label units", "window_metrics": {"available":
true, "median_interpolation_threshold": 60, "window": [54, 57, 60, 63, 66],
"left_flank": 45, "right_flank": 75, "peak_mse": {"0": 171.69461808600684,
"0.0001": 0.9087951740494447, "0.001": 0.5279482555400968, "0.01":
0.45588712125698516}, "unregularized_peak_prominence": 168.06266472
188713, "ridge_peak_reduction": {"0.0001": 170.78582291195738, "0.001":
171.16666983046673, "0.01": 171.23873096474986}}, "paired_comparison":
{"metric": "Test MSE by lambda and feature count", "seeds": [17,
43, 89, 131, 197], "control": {"lambda=0.0001,m=15": [0.609391684
4578774,0.7839348272642679,0.9106690035350387,0.8717941031
295451,0.919200114026833], "lambda=0.0001,m=30": [0.723019133
9488392,0.625509816812327,1.1993765150868427,0.7680989514037853,
0.8376727719423795], "lambda=0.0001,m=45": [3.2003412266548192,
0.8315992826174765,1.8993576778953767,2.4801074834521746,
```

1.395822361965631], "lambda=0.0001,m=54": [10.748629820013699,
3.19831837874375, 8.805513777396342, 3.2711996756319253, 5.6452228357
520235], "lambda=0.0001,m=57": [8.843409940228062, 13.83923511685859,
15.98012283383831, 2.864351384074433, 6.279690466974648], "lambda=
0.0001,m=60": [111.27135274300886, 279.6641804103129, 61.659310172
45382, 394.09176581393183, 11.786481290326796], "lambda=0.0001,m=63":
[64.76815377602193, 84.97058038202381, 9.111914676523618, 6.9592470639
34116, 7.727491058693748], "lambda=0.0001,m=66": [6.292868894092625,
20.100887291344506, 6.499176889059844, 4.732490323505585, 6.1200524212
38772], "lambda=0.0001,m=75": [3.1493425069992873, 4.6923798037
81918, 5.498871927760535, 2.004360836338447, 2.814811745718329]
, "lambda=0.0001,m=90": [2.2264979225225523, 3.734530808724541,
3.5747827125619063, 1.1693572972753243, 2.565359749330958], "lambda=
0.0001,m=120": [1.5097924697287468, 3.1088737804789526, 2.5454374841
217207, 0.8978654041852854, 1.8042944375928929], "lambda=0.0001,m=
180": [1.3623124066170966, 1.6449983557638566, 2.136286757540468,
0.776102503729662, 1.4601973758332], "lambda=0.001,m=15": [0.609391684
4578774, 0.7839348272642679, 0.9106690035350387, 0.8717941031
295451, 0.919200114026833], "lambda=0.001,m=30": [0.723019133
9488392, 0.625509816812327, 1.1993765150868427, 0.7680989514037853,
0.8376727719423795], "lambda=0.001,m=45": [3.2003412266548192,
0.8315992826174765, 1.8993576778953767, 2.4801074834521746,
1.395822361965631], "lambda=0.001,m=54": [10.748629820013699,
3.19831837874375, 8.805513777396342, 3.2711996756319253, 5.6452228357
520235], "lambda=0.001,m=57": [8.843409940228062, 13.83923511685859,
15.98012283383831, 2.864351384074433, 6.279690466974648], "lambda=
0.001,m=60": [111.27135274300886, 279.6641804103129, 61.659310172
45382, 394.09176581393183, 11.786481290326796], "lambda=0.001,m=63":
[64.76815377602193, 84.97058038202381, 9.111914676523618, 6.9592470639
34116, 7.727491058693748], "lambda=0.001,m=66": [6.292868894092625,
20.100887291344506, 6.499176889059844, 4.732490323505585, 6.1200524212
38772], "lambda=0.001,m=75": [3.1493425069992873, 4.6923798037
81918, 5.498871927760535, 2.004360836338447, 2.814811745718329]
, "lambda=0.001,m=90": [2.2264979225225523, 3.734530808724541,
3.5747827125619063, 1.1693572972753243, 2.565359749330958], "lambda=
0.001,m=120": [1.5097924697287468, 3.1088737804789526, 2.5454374841
217207, 0.8978654041852854, 1.8042944375928929], "lambda=0.001,m=
180": [1.3623124066170966, 1.6449983557638566, 2.136286757540468,
0.776102503729662, 1.4601973758332], "lambda=0.01,m=15": [0.609391684
4578774, 0.7839348272642679, 0.9106690035350387, 0.8717941031
295451, 0.919200114026833], "lambda=0.01,m=30": [0.723019133
9488392, 0.625509816812327, 1.1993765150868427, 0.7680989514037853,
0.8376727719423795], "lambda=0.01,m=45": [3.2003412266548192,
0.8315992826174765, 1.8993576778953767, 2.4801074834521746,
1.395822361965631], "lambda=0.01,m=54": [10.748629820013699,
3.19831837874375, 8.805513777396342, 3.2711996756319253, 5.6452228357
520235], "lambda=0.01,m=57": [8.843409940228062, 13.83923511685859,
15.98012283383831, 2.864351384074433, 6.279690466974648], "lambda=
0.01,m=60": [111.27135274300886, 279.6641804103129, 61.659310172
45382, 394.09176581393183, 11.786481290326796], "lambda=0.01,m=63":
[64.76815377602193, 84.97058038202381, 9.111914676523618, 6.9592470639

```
34116, 7.727491058693748], "lambda=0.01, m=66": [6.292868894092625,
20.100887291344506, 6.499176889059844, 4.732490323505585, 6.1200524212
38772], "lambda=0.01, m=75": [3.1493425069992873, 4.6923798037
81918, 5.498871927760535, 2.004360836338447, 2.814811745718329]
, "lambda=0.01, m=90": [2.2264979225225523, 3.734530808724541,
3.5747827125619063, 1.1693572972753243, 2.565359749330958],
"lambda=0.01, m=120": [1.5097924697287468, 3.1088737804789526,
2.5454374841217207, 0.8978654041852854, 1.8042944375928929],
"lambda=0.01, m=180": [1.3623124066170966, 1.6449983557638566,
2.136286757540468, 0.776102503729662, 1.4601973758332]}, "treatment":
{"lambda=0.0001, m=15": [0.6054827339473456, 0.7656940734382196,
0.9093399948091562, 0.8586579599131962, 0.9159172313590412],
"lambda=0.0001, m=30": [0.7146768383180209, 0.6004180397427454,
1.0949913599181955, 0.7101197546361958, 0.7703588676735387],
"lambda=0.0001, m=45": [0.8711015064025922, 0.6188710035832063,
1.1543893680845994, 0.7178205501591737, 0.7674681094057325],
"lambda=0.0001, m=54": [1.168643156584764, 0.637098552349258,
1.1909896193068208, 0.6884042509218063, 0.8588402910845739],
"lambda=0.0001, m=57": [0.947358227283801, 0.6461998031296512,
1.2633557383631406, 0.7245374285298689, 0.8478633011273573],
"lambda=0.0001, m=60": [0.9170417647701462, 0.6540366962419712,
1.2005067084972931, 0.7661730295633397, 0.9039361267159962],
"lambda=0.0001, m=63": [0.8679091445408484, 0.6285523964205181,
1.0945351258465417, 0.8087134215216752, 0.8918885464809797],
"lambda=0.0001, m=66": [0.883979664919387, 0.603868520367972,
1.0690148588127302, 0.7919028995278415, 0.9190390175107325],
"lambda=0.0001, m=75": [0.8554171179438504, 0.7221933303506879,
0.9345721369198969, 0.6988543372249052, 0.9030030623817482],
"lambda=0.0001, m=90": [0.8908149074858861, 0.7002645801790345,
0.9347044131610436, 0.6247786673840369, 1.002937022960439],
"lambda=0.0001, m=120": [0.7642588954136023, 0.6865161293414697,
0.9025601986214102, 0.5669582931633653, 0.8993655730223699],
"lambda=0.0001, m=180": [0.8624565293882164, 0.7044200734920779,
0.9315540169344045, 0.5346142104713495, 0.8097003041046557],
"lambda=0.001, m=15": [0.5810678481210155, 0.6605757984934983,
0.9015269171630846, 0.7713917765927241, 0.8908630809762254],
"lambda=0.001, m=30": [0.6605579136436875, 0.5101676025156539,
0.8116916886209584, 0.5431082606458801, 0.6330595624962365],
"lambda=0.001, m=45": [0.5420367620730941, 0.4822571943750251,
0.7260906315866889, 0.46651870794943745, 0.5749849066768126],
"lambda=0.001, m=54": [0.5048696279661659, 0.47591684703824816,
0.7038630041703043, 0.4230685597476556, 0.5320232387781104],
"lambda=0.001, m=57": [0.45409987118908524, 0.47336438028980465,
0.6792876455413924, 0.4294454122496108, 0.5204740504631309],
"lambda=0.001, m=60": [0.445219755727352, 0.45928599624509153,
0.6453308240116958, 0.4534185077372684, 0.5534432431623894],
"lambda=0.001, m=63": [0.43861706696761543, 0.4511528042435016,
0.603184364032144, 0.45709831054454353, 0.5483694093983419],
"lambda=0.001, m=66": [0.4427534753255407, 0.44386268751320407,
0.6058115551523331, 0.44240795725747595, 0.5544173255939362],
"lambda=0.001, m=75": [0.4353066749258478, 0.4623997667661092,
```

```

0.574701863795957,0.4239678408281918,0.5423375683259061], "lambda=
0.001,m=90": [0.4035340174709676, 0.4464981294598136, 0.5423102644
305188, 0.42351869396566294, 0.5289006523785966], "lambda=0.001,
m=120": [0.38329456697509623, 0.4565833231537416, 0.5104668735
749324, 0.4227912665190684, 0.4825616583321035], "lambda=0.001,
m=180": [0.4405598176506798, 0.45523166425176603, 0.4909992861
570073, 0.40502610729609806, 0.4647213694801534], "lambda=0.01,
m=15": [0.5364640912629668, 0.6119654119243084, 0.9138455668
159602, 0.6073573311537614, 0.8047241100000064], "lambda=0.01,m=
30": [0.571762285532313, 0.4184414580779405, 0.7106172445829515,
0.49390400671304424, 0.49315172708286614], "lambda=0.01,m=45":
[0.5151093133700106, 0.4161566944804978, 0.6189399759665574,
0.45619779485457174, 0.5013412698932961], "lambda=0.01,m=54":
[0.4605588184436772, 0.41168678352693355, 0.5666793448507286,
0.3978768487077916, 0.4426338107557946], "lambda=0.01,m=57":
[0.42712058922845336, 0.41841447837601503, 0.5343873418757948,
0.41023543240667576, 0.4323532573126044], "lambda=0.01,m=60":
[0.41777202117438755, 0.41460918746116326, 0.5236538435697803,
0.41720519124545985, 0.44154014168637223], "lambda=0.01,m=63":
[0.41951472284901586, 0.4132281637281667, 0.5054891015543971,
0.41921922856196764, 0.44337277375546297], "lambda=0.01,m=66":
[0.4145873819003632, 0.41591132245498624, 0.5065008946483052,
0.40587834959621066, 0.44159947065127064], "lambda=0.01,m=75":
[0.39494269472070664, 0.4077360321203164, 0.5018565227263644,
0.4032291434374836, 0.44266767472087787], "lambda=0.01,m=90":
[0.36863465804259554, 0.3981748282977525, 0.4770992860038589,
0.40216855935968715, 0.4277066067864975], "lambda=0.01,m=120":
[0.36292375569823654, 0.3962856967534336, 0.4675832839307553,
0.3867417872481266, 0.4043175479153092], "lambda=0.01,m=180":
[0.36691800746331454, 0.39568831484614553, 0.43795135087454756,
0.3671123275948314, 0.41051241090752766]}, "higher_supports":false,
"control_censored":{"lambda=0.0001,m=15": [false, false, false,
false, false], "lambda=0.0001,m=30": [false, false, false, false, false]
, "lambda=0.0001,m=45": [false, false, false, false, false], "lambda=
0.0001,m=54": [false, false, false, false, false], "lambda=0.0001,m=
57": [false, false, false, false, false], "lambda=0.0001,m=60": [false,
false, false, false, false], "lambda=0.0001,m=63": [false, false, false,
false, false], "lambda=0.0001,m=66": [false, false, false, false, false]
, "lambda=0.0001,m=75": [false, false, false, false, false], "lambda=
0.0001,m=90": [false, false, false, false, false], "lambda=0.0001,m=
120": [false, false, false, false, false], "lambda=0.0001,m=180": [false,
false, false, false, false], "lambda=0.001,m=15": [false, false, false,
false, false], "lambda=0.001,m=30": [false, false, false, false, false],
"lambda=0.001,m=45": [false, false, false, false, false], "lambda=0.001,
m=54": [false, false, false, false, false], "lambda=0.001,m=57": [false,
false, false, false, false], "lambda=0.001,m=60": [false, false, false,
false, false], "lambda=0.001,m=63": [false, false, false, false, false]
, "lambda=0.001,m=66": [false, false, false, false, false], "lambda=
0.001,m=75": [false, false, false, false, false], "lambda=0.001,m=
90": [false, false, false, false, false], "lambda=0.001,m=120": [false,
false, false, false, false], "lambda=0.001,m=180": [false, false, false,

```

```

false, false], "lambda=0.01, m=15": [false, false, false, false, false],
"lambda=0.01, m=30": [false, false, false, false, false], "lambda=0.01,
m=45": [false, false, false, false, false], "lambda=0.01, m=54": [false,
false, false, false, false], "lambda=0.01, m=57": [false, false, false,
false, false], "lambda=0.01, m=60": [false, false, false, false, false],
"lambda=0.01, m=63": [false, false, false, false, false], "lambda=0.01,
m=66": [false, false, false, false, false], "lambda=0.01, m=75": [false,
false, false, false, false], "lambda=0.01, m=90": [false, false, false,
false, false], "lambda=0.01, m=120": [false, false, false, false, false],
"lambda=0.01, m=180": [false, false, false, false, false]}, "treatment_
censored": {"lambda=0.0001, m=15": [false, false, false, false, false]
, "lambda=0.0001, m=30": [false, false, false, false, false], "lambda=
0.0001, m=45": [false, false, false, false, false], "lambda=0.0001, m=
54": [false, false, false, false, false], "lambda=0.0001, m=57": [false,
false, false, false, false], "lambda=0.0001, m=60": [false, false, false,
false, false], "lambda=0.0001, m=63": [false, false, false, false, false]
, "lambda=0.0001, m=66": [false, false, false, false, false], "lambda=
0.0001, m=75": [false, false, false, false, false], "lambda=0.0001, m=
90": [false, false, false, false, false], "lambda=0.0001, m=120": [false,
false, false, false, false], "lambda=0.0001, m=180": [false, false, false,
false, false], "lambda=0.001, m=15": [false, false, false, false, false],
"lambda=0.001, m=30": [false, false, false, false, false], "lambda=0.001,
m=45": [false, false, false, false, false], "lambda=0.001, m=54": [false,
false, false, false, false], "lambda=0.001, m=57": [false, false, false,
false, false], "lambda=0.001, m=60": [false, false, false, false, false],
"lambda=0.001, m=63": [false, false, false, false, false], "lambda=0.001,
m=66": [false, false, false, false, false], "lambda=0.001, m=75": [false,
false, false, false, false], "lambda=0.001, m=90": [false, false, false,
false, false], "lambda=0.001, m=120": [false, false, false, false, false],
"lambda=0.001, m=180": [false, false, false, false, false], "lambda=0.01,
m=15": [false, false, false, false, false], "lambda=0.01, m=30": [false,
false, false, false, false], "lambda=0.01, m=45": [false, false, false,
false, false], "lambda=0.01, m=54": [false, false, false, false, false],
"lambda=0.01, m=57": [false, false, false, false, false], "lambda=0.01,
m=60": [false, false, false, false, false], "lambda=0.01, m=63": [false,
false, false, false, false], "lambda=0.01, m=66": [false, false, false,
false, false], "lambda=0.01, m=75": [false, false, false, false, false],
"lambda=0.01, m=90": [false, false, false, false, false], "lambda=0.01, m=
120": [false, false, false, false, false], "lambda=0.01, m=180": [false,
false, false, false, false]}}, "paired_window_peaks": {"descriptio
n": "Per-repetition maxima in the common median-threshold window;
distinct from maxima of averaged curves.", "seeds": [17, 43, 89,
131, 197], "control": {"0.0001": [111.27135274300886, 279.66418041
03129, 61.65931017245382, 394.09176581393183, 11.786481290326796],
"0.001": [111.27135274300886, 279.6641804103129, 61.65931017245382,
394.09176581393183, 11.786481290326796], "0.01": [111.27135274300886,
279.6641804103129, 61.65931017245382, 394.09176581393183, 11.786481290
326796]}, "treatment": {"0.0001": [1.168643156584764, 0.6540366962
419712, 1.2633557383631406, 0.8087134215216752, 0.9190390175107325]
, "0.001": [0.5048696279661659, 0.47591684703824816, 0.7038630041
703043, 0.45709831054454353, 0.5544173255939362], "0.01": [0.460558818

```

4436772, 0.41841447837601503, 0.5666793448507286, 0.41921922856196764, 0.44337277375546297]], "higher_supports": false, "control_censored": [false, false, false, false, false], "treatment_censored": [false, false, false, false, false]], "interpretation": "C0 > 0 supports the near-threshold peak criterion; D_lambda > 0 supports reduction for that fixed lambda. A finite setting is not a universal result."}

Finding (supports): The results support both components of the conjecture in the implemented Fourier-feature setting. With 60 training observations, all five repetitions reached numerical interpolation at 60 features. The averaged unregularized test MSE peaked at that same feature count: 171.6946, compared with 1.9614 and 3.6320 at the designated flanks of 45 and 75 features. Its recorded near-threshold prominence was 168.0627 squared-label units. Within the common window [54, 57, 60, 63, 66], ridge strengths 0.0001, 0.001, and 0.01 reduced the maximum averaged MSE to 0.9088, 0.5279, and 0.4559, respectively; the corresponding recorded peak reductions were 170.7858, 171.1667, and 171.2387. This is empirical support for the selected setting, not a distribution-independent conclusion. **Uncertainty:** Only five independent repetitions were completed, each evaluated on 2,000 noisy test labels. Unregularized threshold MSE varied substantially across repetitions, from 11.7865 to 394.0918, so the average peak magnitude remains sampling-sensitive. No hypothesis test or confidence interval was supplied. No repetition was discarded, the deadline was not reached, and no numerical failure was reported. Interpolation was defined numerically using a relative residual tolerance of 10^{-8} and an SVD

Robustness: Seeds 17, 43, 89, 131, and 197 all completed. Each unregularized sampled curve attained its maximum at 60 features, and every tested ridge strength reduced the per-repetition maximum within the common threshold window in every seed. Feature counts ranged from 15 to 180, with three fixed positive ridge strengths. Checks were confined to standard Gaussian inputs in six dimensions, a first-coordinate linear target, noise variance 0.2, and Gaussian-spectrum cosine features with bandwidth 2.0. No rob

Outcome confounders:

- The unspecified conjecture was instantiated with one particular target, input distribution, Fourier spectrum, bandwidth, and minimum-norm baseline. These choices may determine whether the observed contrast transfers to other noisy regression settings.
- Peak height and prominence depend on the sampled feature-count grid and the chosen near-threshold window and flanks; unsampled feature counts and alternative window definitions were not evaluated. **Result JSON fields used:** /settings, /completed_seeds, /window_metrics, /average_test_mse/0, /repetitions, /paired_window_peaks, /deadline_reached, /numerical_failure **Artifacts:** result.json, visualization.svg

Exact reusable experiment and visualization code is available on the Experiments page.

```
# RESEARCHPILOT EXPERIMENT PLAN
# BASELINES AND METRICS
# ['Minimum-norm least squares']
# ['Test MSE, in squared-label units: 
$$(E_{\{s, \lambda\}}(m) = \frac{1}{n_{\{\text{test}\}}}) \sum_{i=1}^{n_{\{\text{test}\}}} \left( \phi_{\{s, m\}}(x_{\{\text{test}\}, s, i})^{\top} \widehat{a}_{\{s, m, \lambda\}} - y_{\{\text{te}$$

# WORKING ASSUMPTIONS
# ['User-given conditions: synthetic regression has observation noise; the fitted representation uses random Fourier features; feature count varies; the comparison concerns test error and ridge regulari
```

```

# ALGORITHM STEPS

# ['Start the wall-clock timer and enforce `linear_algebra_threads` and
`arithmetic`.', 'Loop over `seeds`, checking the work-cap deadline before
each expensive operation; discard an unfinished repetiti

# SETTINGS AND RANDOM SEEDS

# {'held_constant': {'n_train': 60, 'n_test': 2000, 'd': 6, 'input_
distribution': 'standard multivariate Gaussian', 'target': 'first input
coordinate', 'noise_variance': 0.2, 'kernel_bandwidth': 2.0, 'p

# [17, 43, 89, 131, 197]

# EXECUTABLE STUDY

# Pure-stdlib float64 random Fourier regression experiment.

# Direct Golub-Reinsch thin SVD (bidiagonalization + implicit QR), not

# normal-equation eigendecomposition. Fixed ridge values; no test
selection.

import time

START = time.monotonic()

DEADLINE = START + 280.0

import os

for name in ('OMP_NUM_THREADS', 'OPENBLAS_NUM_THREADS', 'MKL_NUM_THREADS',
'BLIS_NUM_THREADS', 'VECLIB_MAXIMUM_THREADS', 'NUMEXPR_NUM_THREADS'):

    os.environ[name] = '1'

import math, random, json, statistics, struct, sys
assert struct.calcsize('d') == 8 and sys.float_info.mant_dig == 53
SEEDS = [17, 43, 89, 131, 197]
GRID = [15, 30, 45, 54, 57, 60, 63, 66, 75, 90, 120, 180]
LAMBDA_S = [0.0, 0.0001, 0.001, 0.01]
KEYS = ['0', '0.0001', '0.001', '0.01']
N, NT, DIM, MAXM = 60, 2000, 6, 180
RHO, TOL = 1e-12, 1e-8

class Deadline(Exception): pass

def check():

    if time.monotonic() >= DEADLINE: raise Deadline()

def dot(a,b): return sum(x*y for x,y in zip(a,b))

def sign(a,b): return abs(a) if b >= 0.0 else -abs(a)

def transpose(a): return [list(c) for c in zip(*a)]

def tall_svd(a):

    # Golub-Reinsch SVD for rows >= columns; returns U, singular values,
    V.

    a = [row[:] for row in a]
    nr, nc = len(a), len(a[0])

```

```

w = [0.0]*nc
rv = [0.0]*nc
v = [[0.0]*nc for _ in range(nc)]
g = scale = anorm = 0.0
for i in range(nc):
    check()
    l = i+1
    rv[i] = scale*g
    g = scale = s = 0.0
    scale = sum(abs(a[k][i]) for k in range(i,nr))
    if scale:
        for k in range(i,nr):
            a[k][i] /= scale
            s += a[k][i]*a[k][i]
        f = a[i][i]
        g = -sign(math.sqrt(s),f)
        h = f*g-s
        a[i][i] = f-g
        for j in range(l,nc):
            s = sum(a[k][i]*a[k][j] for k in range(i,nr))
            f = s/h
            for k in range(i,nr): a[k][j] += f*a[k][i]
        for k in range(i,nr): a[k][i] *= scale
    w[i] = scale*g
    g = scale = s = 0.0
    if i < nc-1:
        scale = sum(abs(a[i][k]) for k in range(l,nc))
        if scale:
            for k in range(l,nc):
                a[i][k] /= scale
                s += a[i][k]*a[i][k]
            f = a[i][l]
            g = -sign(math.sqrt(s),f)
            h = f*g-s
            a[i][l] = f-g
            for k in range(l,nc): rv[k] = a[i][k]/h
            for j in range(l,nr):
                s = sum(a[j][k]*a[i][k] for k in range(l,nc))

```

```

        for k in range(l,nc): a[j][k] += s*rv[k]
        for k in range(l,nc): a[i][k] *= scale
        anorm = max(anorm,abs(w[i])+abs(rv[i]))
# Accumulate right Householder transformations.
g = 0.0
for i in range(nc-1,-1,-1):
    l = i+1
    if i < nc-1:
        if g:
            for j in range(l,nc): v[j][i] = (a[i][j]/a[i][l])/g
            for j in range(l,nc):
                s = sum(a[i][k]*v[k][j] for k in range(l,nc))
                for k in range(l,nc): v[k][j] += s*v[k][i]
            for j in range(l,nc): v[i][j] = v[j][i] = 0.0
        v[i][i] = 1.0
        g = rv[i]
# Accumulate left Householder transformations.
for i in range(nc-1,-1,-1):
    l = i+1
    g = w[i]
    for j in range(l,nc): a[i][j] = 0.0
    if g:
        inv = 1.0/g
        for j in range(l,nc):
            s = sum(a[k][i]*a[k][j] for k in range(l,nr))
            f = (s/a[i][i])*inv
            for k in range(i,nr): a[k][j] += f*a[k][i]
        for j in range(i,nr): a[j][i] *= inv
    else:
        for j in range(i,nr): a[j][i] = 0.0
    a[i][i] += 1.0
# Diagonalize the bidiagonal matrix by implicit QR.
eps = sys.float_info.epsilon
for k in range(nc-1,-1,-1):
    for iteration in range(100):
        check()
        flag = True
        for l in range(k,-1,-1):

```

```

        nm = l-1
        if abs(rv[l]) <= eps*anorm:
            flag = False
            break
        if nm >= 0 and abs(w[nm]) <= eps*anorm: break
    if flag:
        c, s = 0.0, 1.0
        for i in range(l, k+1):
            f = s*rv[i]
            rv[i] = c*rv[i]
            if abs(f) <= eps*anorm: break
            g = w[i]
            h = math.hypot(f, g)
            w[i] = h
            c, s = g/h, -f/h
            for j in range(nr):
                y, z = a[j][nm], a[j][i]
                a[j][nm], a[j][i] = y*c+z*s, z*c-y*s
        z = w[k]
        if l == k:
            if z < 0.0:
                w[k] = -z
                for j in range(nc): v[j][k] = -v[j][k]
            break
    if iteration == 99: raise ArithmeticError('SVD QR did not
converge')

    x = w[l]
    nm = k-1
    y, g, h = w[nm], rv[nm], rv[k]
    f = ((y-z)*(y+z)+(g-h)*(g+h))/(2.0*h*y)
    g = math.hypot(f, 1.0)
    f = ((x-z)*(x+z)+h*((y/(f+sign(g, f)))-h))/x
    c = s = 1.0
    for j in range(l, nm+1):
        i = j+1
        g, y = rv[i], w[i]
        h = s*g
        g = c*g
        z = math.hypot(f, h)

```

```

        rv[j] = z
        c, s = f/z, h/z
        f, g = x*c+g*s, g*c-x*s
        h = y*s
        y *= c
        for jj in range(nc):
            x, z = v[jj][j], v[jj][i]
            v[jj][j], v[jj][i] = x*c+z*s, z*c-x*s
        z = math.hypot(f, h)
        w[j] = z
        if z: c, s = f/z, h/z
        f, x = c*g+s*y, c*y-s*g
        for jj in range(nr):
            y, z = a[jj][j], a[jj][i]
            a[jj][j], a[jj][i] = y*c+z*s, z*c-y*s
        rv[l], rv[k], w[k] = 0.0, f, x
    order = sorted(range(nc), key=lambda i:w[i], reverse=True)
    return ([[row[i] for i in order] for row in a],
            [w[i] for i in order], [[row[i] for i in order] for row in v])

def svd(a):
    check()
    if len(a) >= len(a[0]): return tall_svd(a)
    u, s, v = tall_svd(transpose(a))
    return v, s, u

def prefix(c, m):
    check()
    q = math.sqrt(2.0/m)
    return [[q*z for z in row[:m]] for row in c]

def project_y(u, y): return [dot(col, y) for col in zip(*u)]

def coefficients(v, s, uy, lam):
    check()
    if lam == 0.0:
        factors = [b/z if z > RHO*s[0] else 0.0 for z, b in zip(s, uy)]
    else:
        factors = [b*z/(z*z+N*lam) for z, b in zip(s, uy)]
    return [dot(row, factors) for row in v]

def residual(a, coef, y):
    return math.sqrt(sum((dot(row, coef)-t)**2 for row, t in zip(a, y)))/

```

```

dot(y, y))
def mse(a, coef, y):
    total = 0.0
    for i, (row, t) in enumerate(zip(a, y)):
        if i % 100 == 0: check()
        e = dot(row, coef) - t
        total += e * e
    val = total / len(y)
    if not math.isfinite(val): raise ArithmeticError('nonfinite MSE')
    return val

def repetition(seed):
    check()
    rng = random.Random(seed)
    x = [[rng.gauss(0.0, 1.0) for _ in range(DIM)] for _ in range(N)]
    xt = [[rng.gauss(0.0, 1.0) for _ in range(DIM)] for _ in range(NT)]
    tau = math.sqrt(0.2)
    y = [row[0] + rng.gauss(0.0, tau) for row in x]
    yt = [row[0] + rng.gauss(0.0, tau) for row in xt]
    # Draws occur after data/noise, from disjoint independent PRNG draws.
    omega = [[rng.gauss(0.0, 0.5) for _ in range(DIM)] for _ in
range(MAXM)]
    phase = [rng.uniform(0.0, 2.0 * math.pi) for _ in range(MAXM)]
    def cache(rows):
        out = []
        for i, row in enumerate(rows):
            if i % 50 == 0: check()
            out.append([math.cos(dot(row, w) + b) for w, b in zip(omega,
phase)])
        return out
    c, ct = cache(x), cache(xt)
    scan = []
    threshold = None
    for m in range(1, 181):
        a = prefix(c, m)
        u, s, v = svd(a)
        uy = project_y(u, y)
        coef = coefficients(v, s, uy, 0.0)
        r = residual(a, coef, y)
        check()

```

```

        scan.append([m,r,sum(z > RHO*s[0] for z in s)])
    if r <= TOL:
        threshold = m
        break
errors = {key:[] for key in KEYS}
train_residuals = []
singular_extremes = []
for m in GRID:
    a,at = prefix(c,m),prefix(ct,m)
    u,s,v = svd(a)
    uy = project_y(u,y)
    singular_extremes.append([s[0],s[-1]])
    for lam,key in zip(LAMBDAS,KEYS):
        coef = coefficients(v,s,uy,lam)
        if lam == 0.0: train_residuals.append(residual(a,coef,y))
        errors[key].append(mse(at,coef,yt))
    check()
    return {'seed':seed,'threshold':threshold if threshold is not None
else 'not reached',
        'threshold_censored':threshold is None,'threshold_scan_m_
residual_rank':scan,
        'test_mse':errors,'baseline_relative_train_residual':train_
residuals,
        'curve_singular_max_min':singular_extremes,
        'work_counts':{'threshold_svd_calls':len(scan),'curve_svd_
calls':12,'curve_coefficient_fits':48}}

completed = []
stopped = False
failure = None
for seed in SEEDS:
    try:
        check()
        record = repetition(seed)
        completed.append(record)
    except Deadline:
        stopped = True
        break
    except (ArithmeticError,OverflowError,ZeroDivisionError) as exc:
        failure = {'seed':seed,'error':str(exc)}

```

```

        break
S = len(completed)
averages = {k:[sum(r['test_mse'][k][j] for r in completed)/S for j in
range(len(GRID))] for k in KEYS} if S else {}
threshold_median = statistics.median([r['threshold'] for r in completed])
if S and all(not r['threshold_censored'] for r in completed) else None
window = [m for m in GRID if abs(m/threshold_median-1.0) <= 0.1+1e-14] if
threshold_median else []
left = max((m for m in GRID if window and m < min(window)),default=None)
right = min((m for m in GRID if window and m > max(window)),default=None)
valid = bool(window and left is not None and right is not None)
summary = {'available':valid,'median_interpolation_threshold':threshold_
median,
           'window':window,'left_flank':left,'right_flank':right,
           'peak_mse':None,'unregularized_peak_prominence':None,'ridge_
peak_reduction':None}
if valid:
    idx = [GRID.index(m) for m in window]
    peaks = {k:max(averages[k][j] for j in idx) for k in KEYS}
    prominence = peaks['0']-max(averages['0'][GRID.index(left)],
averages['0'][GRID.index(right)])
    summary.update(peak_mse=peaks,unregularized_peak_prominence=
prominence,
                  ridge_peak_reduction={k:peaks['0']-peaks[k] for k in
KEYS[1:]})
else:
    summary['unavailable_reason'] = 'No completed repetitions, at least
one threshold not reached, empty window, or missing flank.'
# Primary paired comparisons at every grid count. All arrays share
completed-seed order.
# Lower treatment MSE supports peak reduction; these raw comparisons are
not
# substituted for max-of-averaged-curve window statistics.
control,treatment = {},{}
for k in KEYS[1:]:
    for j,m in enumerate(GRID):
        regime = 'lambda='+k+',m='+str(m)
        control[regime] = [r['test_mse']['0'][j] for r in completed]
        treatment[regime] = [r['test_mse'][k][j] for r in completed]
paired_peaks = None
if valid:
    paired_peaks = {'description':'Per-repetition maxima in the common

```

```

median-threshold window; distinct from maxima of averaged curves.',
        'seeds':[r['seed'] for r in completed],
        'control':{k:[max(r['test_mse']['0'][j] for j in idx)
for r in completed] for k in KEYS[1:]},
        'treatment':{k:[max(r['test_mse'][k][j] for j in idx)
for r in completed] for k in KEYS[1:]},
        'higher_supports':False, 'control_censored':[False]*S,
'treatment_censored':[False]*S}
result = {
    'experiment':'Fourier-feature interpolation peak and ridge
comparison',
    'method':'Gaussian-spectrum cosine features; direct Golub-Reinsch
thin SVD; cutoff minimum-norm least squares and untruncated SVD ridge',
    'source_scope':'The supplied paper example uses ReLU and feature-
dependent optimal ridge; this is the explicitly designed Fourier
experiment with fixed ridge strengths, not a replication of that
example.',
    'settings':{'n_train':N, 'n_test':NT, 'd':DIM, 'input_distribution':
'standard multivariate Gaussian',
        'target':'first input coordinate', 'noise_variance':0.2,
'kernel_bandwidth':2.0,
        'phase_distribution':'uniform on [0, 2pi)', 'feature_
sequence':'nested prefixes within each repetition',
        'test_labels':'independent noisy labels', 'linear_algebra_
threads':1, 'arithmetic':'float64',
        'svd_relative_cutoff':RHO, 'interpolation_residual_
tolerance':TOL, 'near_threshold_fraction':0.1,
        'feature_counts':GRID, 'ridge_parameters':LAMBDA_S[1:],
'baseline_regularization':0,
        'ridge_objective':'||Phi a-y||^2/n_train + lambda
||a||^2',
        'threshold_scan':{'first_feature_count':1, 'last_feature_
count':180, 'increment':1},
        'work_cap':{'wall_clock_seconds':280, 'maximum_features_
per_repetition':180,
            'maximum_threshold_svd_calls_per_repetition':
180, 'maximum_curve_svd_calls_per_repetition':12,
            'maximum_coefficient_fits_per_repetition':48},
        'rng':'stdlib random.Random, Gaussian draws via gauss;
one independent stream per seed'},
    'requested_seeds':SEEDS, 'completed_seeds':[r['seed'] for r in
completed],
    'completed_repetition_count':S, 'discarded_or_unstarted_seeds':
SEEDS[S:],
    'deadline_reached':stopped, 'numerical_failure':failure,

```

```

    'elapsed_seconds':time.monotonic()-START, 'repetitions':completed,
    'average_test_mse':averages, 'mse_units':'squared-label units',
    'window_metrics':summary,
    'paired_comparison':{'metric':'Test MSE by lambda and feature count',
    'seeds':[r['seed'] for r in completed],
                        'control':control, 'treatment':treatment, 'higher_
supports':False,
                        'control_censored':{'k:[False]*S for k in
control}},
                        'treatment_censored':{'k:[False]*S for k in
treatment}}},
    'paired_window_peaks':paired_peaks,
    'interpretation':'C0 > 0 supports the near-threshold peak criterion;
D_lambda > 0 supports reduction for that fixed lambda. A finite setting is
not a universal result.}'
encoded = json.dumps(result,ensure_ascii=False,allow_nan=False,
separators=(',', ':')).encode('utf-8')
assert len(encoded) <= 100000
with open('result.json','wb') as f: f.write(encoded)

```

Visualization code (reads result.json):

```

# Static stdlib SVG; measurements are loaded only from result.json.
import json, math
from xml.sax.saxutils import escape
with open('result.json',encoding='utf-8') as f: r = json.load(f)
W,H = 1040,740

parts = ['<svg xmlns="http://www.w3.org/2000/svg" width="1040" height=
"740" viewBox="0 0 1040 740" role="img" aria-label="Random Fourier
feature regression test error curves">',

        '<title>Fourier-feature interpolation and fixed ridge
comparison</title>',

        '<desc>Mean test MSE over completed seed repetitions. Vertical
axis is logarithmic. Faint curves show individual repetitions. Window
statistics use maxima of averaged curves.</desc>',

        '<rect x="0" y="0" width="1040" height="740" fill="white"/>']

def text(x,y,t,size=14,color='#222',anchor='start'):
    parts.append('<text x="%g" y="%g" font-family="sans-serif" font-size=
"%g" fill="%s" text-anchor="%s">%s</text>'%(x,y,size,color,anchor,
escape(str(t))))

def line(x1,y1,x2,y2,color='#bbb',width=1,dash=None):
    extra = ' stroke-dasharray="%s"%dash if dash else ''

    parts.append('<line x1="%g" y1="%g" x2="%g" y2="%g" stroke="%s"
stroke-width="%g"%s/>'%(x1,y1,x2,y2,color,width,extra))

text(65,32,'Random Fourier features: interpolation peak and ridge',23)

```

```

S = r['completed_repetition_count']

text(65,57,'Completed repetitions: %s / %s; n_train = %s; independent
noisy test labels'%(S,len(r['requested_seeds']),r['settings']['n_train'])
,14)

if not S:

    text(65,120,'No completed repetitions: no measured curves are
available.',18)

else:

    grid = r['settings']['feature_counts']
    curves = r['average_test_mse']
    keys = ['0','0.0001','0.001','0.01']
    colors = ['#222222','#0072b2','#d55e00','#009e73']

    allvals = [v for rec in r['repetitions'] for arr in rec['test_mse']
.values() for v in arr]

    # A log axis requires strictly positive MSE; measured MSE is positive
here.

    positive = [v for v in allvals if v > 0]

    if not positive: raise ValueError('Log-axis plot requires positive
measured MSE')

    lo = math.floor(math.log10(min(positive)))
    hi = math.ceil(math.log10(max(positive)))
    if hi == lo: hi += 1
    x0,y0,pw,ph = 90,90,740,410

    def X(m): return x0+(m-min(grid))/(max(grid)-min(grid))*pw
    def Y(v): return y0+ph-(math.log10(v)-lo)/(hi-lo)*ph

    wm = r['window_metrics']
    win = wm['window']

    if win:

        xa,xb = X(min(win)),X(max(win))

        parts.append('<rect x="%g" y="%g" width="%g" height="%g" fill=
"#eee9bf" fill-opacity="0.65"/>'%(xa,y0,max(xb-xa,1),ph))

        for exp in range(lo,hi+1):

            yy = Y(10.0**exp)
            line(x0,yy,x0+pw,yy,'#dddddd')
            text(x0-10,yy+5,'%g'%(10.0**exp),12,anchor='end')

        for m in grid:

            xx = X(m)
            line(xx,y0+ph,xx,y0+ph+5,'#555')
            text(xx,y0+ph+20,m,10,anchor='middle')

        line(x0,y0,x0,y0+ph,'#555')
        line(x0,y0+ph,x0+pw,y0+ph,'#555')

```

```

text(x0+pw/2, 547, 'Feature count m (features)', 15, anchor='middle')

parts.append('<text x="25" y="295" transform="rotate(-90 25 295)"
font-family="sans-serif" font-size="14" fill="#222" text-anchor=
"middle">Test MSE (squared-label units; log scale)</text>')

med = wm['median_interpolation_threshold']

if med is not None:
    xx = X(med)
    line(xx, y0, xx, y0+ph, '#777', 1.5, '5 4')
    text(xx+6, y0+18, 'median interpolation: %g'%med, 12)

for key, color in zip(keys, colors):
    for rec in r['repetitions']:
        vals = rec['test_mse'][key]
        points = ' '.join('%g,%g'%(X(m), Y(v)) for m, v in zip(grid,
vals) if v>0)

        parts.append('<polyline points="%s" fill="none" stroke="%s"
stroke-width="1" stroke-opacity="0.18"/>'%(points, color))

        vals = curves[key]
        points = ' '.join('%g,%g'%(X(m), Y(v)) for m, v in zip(grid, vals)
if v>0)

        parts.append('<polyline points="%s" fill="none" stroke="%s"
stroke-width="2.5"/>'%(points, color))

        for m, v in zip(grid, vals):
            if v>0: parts.append('<circle cx="%g" cy="%g" r="3" fill="%s"/
>'%(X(m), Y(v), color))

    for i, (key, color) in enumerate(zip(keys, colors)):
        yy = 123+i*38
        line(850, yy, 879, yy, color, 3)
        text(886, yy+5, 'LS' if key=='0' else 'ridge '+key, 13)
    text(850, 300, 'Bold: means', 12)
    text(850, 320, 'Faint: seeds', 12)
    text(850, 345, 'Shaded: window', 12)

    text(65, 580, 'Numerical thresholds by seed: '+', '.join('%s:
%s'%(rec['seed'], rec['threshold']) for rec in r['repetitions']), 13)

    if wm['available']:
        text(65, 605, 'Window: %s; adjacent flanks: %s, %s features'%(win,
wm['left_flank'], wm['right_flank']), 13)

        text(65, 630, 'Unregularized peak P0 = %.6g; prominence C0 = %.6g
squared-label units'%(wm['peak_mse']['0'], wm['unregularized_peak_
prominence']), 13)

        reductions = '; '.join('lambda %s: %.6g'%(k, wm['ridge_peak_
reduction'][k]) for k in keys[1:])

        text(65, 655, 'Peak reductions D_lambda = P0 - P_lambda:
'+reductions, 13)

```

```

else:
    text(65,615,'Window metrics unavailable: '+wm.get('unavailable_
reason',''),12)
text(65,701,'Fixed ridge parameters, not optimized on test data. One
finite setting does not establish a universal claim.',12)
parts.append('</svg>')
svg = '\n'.join(parts)
assert len(svg.encode('utf-8')) <= 1000000
with open('visualization.svg','w',encoding='utf-8') as f: f.write(svg)

```

4.2 Potential counterexamples

- None recorded.

5 Interpretation

For the tested Gaussian-input, first-coordinate linear regression setting with noise variance 0.2, 60 training observations, and normalized Gaussian-spectrum cosine features of bandwidth 2.0, the five-repetition average noisy-label test MSE on the sampled feature-count grid peaks at the observed numerical interpolation threshold of 60 features. Under the implemented ridge normalization, each tested strength $\lambda \in \{0.0001, 0.001, 0.01\}$ lowers the maximum averaged MSE within the sampled 54–66-feature window relative to minimum-norm least squares. This is an empirical statement about the tested setting.

6 What remains uncertain

- The claim does not specify whether it is intended as a typical empirical phenomenon, a statement about one particular synthetic setting, or a universal assertion over noisy regression problems.
- The supplied source passages concern ReLU random features, not a defined random Fourier feature method. Their conclusions cannot silently be transferred to Fourier features, and the passages do not supply a complete alternative algorithm.
- The synthetic target, input distribution, noise distribution and variance, kernel or spectral distribution, bandwidth, dimensions, and sample sizes are unspecified.
- The unregularized fitting convention is unspecified when least-squares solutions are nonunique; the minimum-norm convention above is an explicit working choice.
- Ridge reduction could mean reduction under a fixed positive parameter, a suitably chosen parameter, or feature-count-dependent optimal regularization. The source illustrates optimal regularization, but the user does not require that interpretation or claim improvement for every positive parameter.
- The meanings of ‘near’ and ‘peak’ require a numerical window and a peak criterion. Numerical interpolation also requires a residual tolerance.
- Test error may refer to error against noisy labels or the noiseless target, and may be conditional on one training sample or averaged over data and features. The proposed measurement uses noisy independent labels and repeated averages.

- Does the claim concern a particular setting, typical realizations, population-averaged risk, or all noisy Fourier-feature regression problems?
- Do substantially more repetitions preserve the peak location and ridge contrast, and how uncertain is the average peak magnitude?
- How do the findings change with the input distribution, target, noise law and level, spectral distribution, bandwidth, dimension, and training sample size?
- Does suppression persist for other ridge strengths, including very weak or strong penalties, and for independently validated feature-dependent choices?
- Would a denser test-error grid near interpolation or alternative prespecified windows and flanks change the peak conclusions?
- Can implementation inspection verify the planned feature normalization and Gaussian-noise generation, and can numerical sensitivity checks confirm stability under alternative SVD cutoffs and precision?
- What Fourier-feature-specific theorem or inspected experiment establishes conditions for these effects beyond this finite setting, and how do results differ when evaluated against the noiseless target?

7 Sources

- [The generalization error of random features regression: Precise asymptotics and double descent curve.](#)
- [Multiple Descent in the Multiple Random Feature Model.](#)
- [High-Dimensional Asymptotics of Prediction: Ridge Regression and Classification.](#)
- [Analysis of Interpolating Regression Models and the Double Descent Phenomenon.](#)

8 Usage

Live model requests: 10. Reused model responses: 0. All cache hits: 0. Tokens: 246031 input, 17151 output (263182 total).

Estimated provider cost: \$0.683572 USD estimated.

- interpretation (openai:gpt-6.1-sol): 4064 input, 2204 output tokens; completed; \$0.030168 estimated.
- ai_general_reasoning (openai:gpt-6.1-sol): 35875 input, 1502 output tokens; completed; \$0.106770 estimated.
- additional_literature_search (openai:gpt-6.1-sol): 10826 input, 81 output tokens; completed; \$0.022462 estimated.
- source_selection (openai:gpt-6.1-sol): 15524 input, 174 output tokens; completed; \$0.032788 estimated.
- evidence_extraction (openai:gpt-6.1-sol): 17239 input, 1497 output tokens; completed; \$0.049448 estimated.
- experiment_planning (openai:gpt-6.1-sol): 3303 input, 2181 output tokens; completed; \$0.028416 estimated.
- experiment_code (openai:gpt-6.1-sol): 18160 input, 6744 output tokens; completed; \$0.103760 estimated.
- experiment_execution (openai:gpt-6.1-sol): 54621 input, 809 output tokens; completed;

\$0.117332 estimated.

- evidence_synthesis (openai:gpt-6.1-sol): 42863 input, 1100 output tokens; completed; \$0.096726 estimated.
- confidence_estimation (openai:gpt-6.1-sol): 43556 input, 859 output tokens; completed; \$0.095702 estimated.